

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Code:

```
geya@LenovoL480: ~/OS/Pra x + v
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int source, destination;
    char buffer[100];
    int n;

    source = open("source.txt", O_RDONLY);
    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (source < 0 || destination < 0) {
        printf("Error opening file\n");
        return 1;
    }

    while ((n = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, n);
    }

    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 2$ gcc Task_1.c -o Task_1
geya@LenovoL480:~/OS/Practical/Week 2$ ./Task_1
File copied successfully.
geya@LenovoL480:~/OS/Practical/Week 2$ cat copy.txt
Hello, this is a test file.
geya@LenovoL480:~/OS/Practical/Week 2$ |
```

System calls provide an interface between user-space programs and kernel-space services. The `open()`, `read()`, `write()`, and `close()` calls allow the program to request file operations from the kernel.